Version control

Version control (also known as **revision control**, **source control**, and **source code management**) is the software engineering practice of controlling, organizing, and tracking different versions in history of computer files; primarily source code text files, but generally any type of file.

Version control is a component of software configuration management.[1]

A *version control system* is a software tool that automates version control. Alternatively, version control is embedded as a feature of some systems such as word processors, spreadsheets, collaborative web docs,[2] and content management systems, such as Wikipedia's page history.

Overview

As teams develop product, it is common to deploy multiple versions of the same software, and for different developers to work on one or more different versions simultaneously. Bugs or features of the software are often only present in certain versions (because of the fixing of some problems and the introduction of others as the program develops). Therefore, for the purposes of locating and fixing bugs, it is vitally important to be able to retrieve and run different versions of the software to determine in which version(s) the problem occurs. It may also be necessary to develop two versions of the software concurrently: for instance, where one version has bugs fixed, but no new features (branch), while the other version is where new features are worked on (trunk).

At the simplest level, developers could simply retain multiple copies of the different versions of the program, and label them appropriately. This simple approach has been used in many large software projects. While this method can work, it is inefficient as many near-identical copies of the program have to be maintained. This requires a lot of self-discipline on the part of developers and often leads to mistakes. Since the code base is the same, it also requires granting read-write-execute permission to a set of developers, and this adds the pressure of someone managing permissions so that the code base is not compromised, which adds more complexity. Consequently, systems to automate some or all of the revision control process have been developed. This abstracts most operational steps (hides them from ordinary users).

Moreover, in software development, legal and business practice, and other environments, it has become increasingly common for a single document or snippet of code to be edited by a team, the members of which may be geographically dispersed and may pursue different and even contrary interests. Sophisticated revision control that tracks and accounts for ownership of changes to documents and code may be extremely helpful or even indispensable in such situations.

Revision control may also track changes to configuration files, such as those typically stored in `/etc` or `/usr/local/etc` on Unix systems. This gives system administrators another way to easily track changes made and a way to roll back to earlier versions should the need arise.

Many version control systems identify the version of a file as a number or letter, called the *version number*, *version*, *revision number*, *revision*, or *revision level*. For example, the first version of a file might be version *1*. When the file is changed the next version is *2*. Each version is associated with a timestamp and the person making the change. Revisions can be compared, restored, and, with some types of files, merged.[3]

History

IBM's OS/360 IEBUPDTE software update tool dates back to 1962, arguably a precursor to version control system tools. Two source management and version control packages that were heavily used by IBM 360/370 installations were The Librarian and Panvalet.[4][5]

A full system designed for source code control was started in 1972: the Source Code Control System (SCCS), again for the OS/360. SCCS's user manual, especially the introduction, which was published on December 4, 1975, implied that it was the first deliberate revision control system.[6] The Revision Control System (RCS) followed in 1982[7] and, later, Concurrent Versions System (CVS) added network and concurrent development features to RCS. After CVS, a dominant successor was Subversion,[8] followed by the rise of distributed version control tools such as Git.[9]

Structure

Revision control manages changes to a set of data over time. These changes can be structured in various ways.

Often the data is thought of as a collection of many individual items, such as files or documents, and changes to individual files are tracked. This is in line with the notion of keeping files separate but causes problems when identity changes, which happens when files are renamed, split or merged. Accordingly, some systems such as Git, consider changes to the data as a whole instead, which is less intuitive for simple changes but simplifies more complex changes.

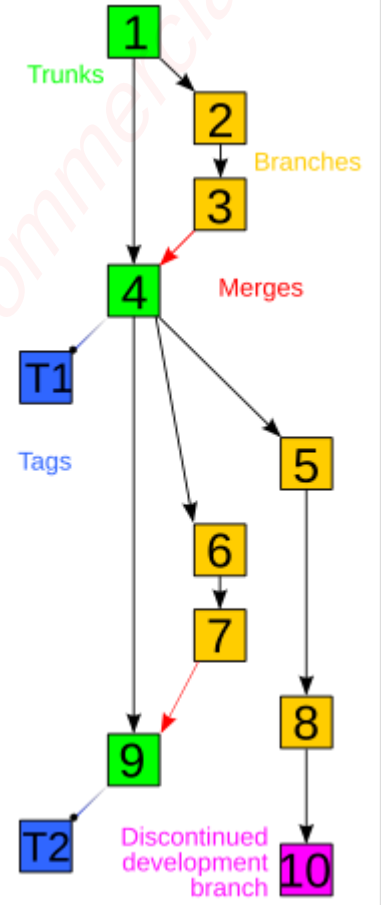
When data that is under revision control is modified, after being retrieved by *checking out*, this is not in general immediately reflected in the revision control system (in the *repository*), but must instead be *checked in* or *committed*. A copy outside of revision control is known as a "working copy". As a simple example, when editing a computer file, the data stored in memory by the editing program is the working copy, which is committed by saving the file. Concretely, one may print out a document, edit it by hand, and only later manually input the changes into a computer and save it. For source code control, the working copy is instead a copy of all files in a particular revision, generally stored locally on the developer's computer;[note 1] in this case saving the file only changes the working copy, and checking into the repository is a separate step.

If multiple people are working on a single data set or document, they are implicitly creating branches of the data (in their working copies), and thus issues of merging arise, as discussed below. For simple collaborative document editing, this can be prevented by using file locking or simply avoiding working on the same document that someone else is working on.

Revision control systems are often centralized, with a single authoritative data store, the *repository*, and check-outs and check-ins done with reference to this central repository. Alternatively, in distributed revision control, no single repository is authoritative, and data can be checked out and checked into any repository. When checking into a different repository, this is interpreted as a merge or patch.

Graph structure

In terms of graph theory, revisions are generally thought of as a line of development (the *trunk*) with branches off of this, forming a directed tree, visualized as one or more parallel lines of development (the "mainlines" of the branches) branching off a trunk. In reality the structure is more complicated, forming a directed acyclic graph, but for many purposes "tree with merges" is an adequate approximation.



Example history graph of a revision-controlled project, with the trunk in green and the branches in yellow; the graph is not a tree, however, due to presence of merges (the red arrows).

Revisions occur in sequence over time, and thus can be arranged in order, either by revision number or timestamp.[note 2] Revisions are based on past revisions, though it is possible to largely or completely replace an earlier revision, such as "delete all existing text, insert new text". In the simplest case, with no branching or undoing, each revision is based on its immediate predecessor alone, and they form a simple line, with a single latest version, the "HEAD" revision or *tip*. In graph theory terms, drawing each revision as a point and each "derived revision" relationship as an arrow (conventionally pointing from older to newer, in the same direction as time), this is a linear graph. If there is branching, so multiple future revisions are based on a past revision, or undoing, so a revision can depend on a revision older than its immediate predecessor, then the resulting graph is instead a directed tree (each node can have more than one child), and has multiple tips, corresponding to the revisions without children ("latest revision on each branch").[note 3] In principle the resulting tree need not have a preferred tip ("main" latest revision) – just various different revisions – but in practice one tip is generally identified as HEAD. When a new revision is based on HEAD, it is either identified as the new HEAD, or considered a new branch.[note 4] The list of revisions from the start to HEAD (in graph theory terms, the unique path in the tree, which forms a linear graph as before) is the *trunk* or *mainline*.[note 5] Conversely, when a revision can be based on more than one previous revision (when a node can have more than one *parent*), the resulting process is called a *merge*, and is one of the most complex aspects of revision control. This most often occurs when changes occur in multiple branches (most often two, but more are possible), which are then merged into a single branch incorporating both changes. If these changes overlap, it may be difficult or impossible to merge, and require manual intervention or rewriting.

In the presence of merges, the resulting graph is no longer a tree, as nodes can have multiple parents, but is instead a rooted directed acyclic graph (DAG). The graph is acyclic since parents are always backwards in time, and rooted because there is an oldest version. Assuming there is a trunk, merges from branches can be considered as "external" to the tree – the changes in the branch are packaged up as a *patch*, which

is applied to HEAD (of the trunk), creating a new revision without any explicit reference to the branch, and preserving the tree structure. Thus, while the actual relations between versions form a DAG, this can be considered a tree plus merges, and the trunk itself is a line.

In distributed revision control, in the presence of multiple repositories these may be based on a single original version (a root of the tree), but there need not be an original root - instead there can be a separate root (oldest revision) for each repository. This can happen, for example, if two people start working on a project separately. Similarly, in the presence of multiple data sets (multiple projects) that exchange data or merge, there is no single root, though for simplicity one may think of one project as primary and the other as secondary, merged into the first with or without its own revision history.

Specialized strategies

Engineering revision control developed from formalized processes based on tracking revisions of early blueprints or bluelines. This system of control implicitly allowed returning to an earlier state of the design, for cases in which an engineering dead-end was reached in the development of the design. A revision table was used to keep track of the changes made. Additionally, the modified areas of the drawing were highlighted using revision clouds.

In business and law

Version control is widespread in business and law. Indeed, "contract redline" and "legal blackline" are some of the earliest forms of revision control,[10] and are still employed in business and law with varying degrees of sophistication. The most sophisticated techniques are beginning to be used for the electronic tracking of changes to CAD files (see product data management), supplanting the "manual" electronic implementation of traditional revision control.

Source-management models

Traditional revision control systems use a centralized model where all the revision control functions take place on a shared server. If two developers try to change the same file at the same time, without some method of managing access the developers may end up overwriting each other's work. Centralized revision control systems solve this problem in one of two different "source management models": file locking and version merging.

Atomic operations

An operation is *atomic* if the system is left in a consistent state even if the operation is interrupted. The *commit* operation is usually the most critical in this sense. Commits tell the revision control system to make a group of changes final, and available to all users. Not all revision control systems have atomic commits; Concurrent Versions System lacks this feature.[11]

File locking

The simplest method of preventing "concurrent access" problems involves locking files so that only one developer at a time has write access to the central "repository" copies of those files. Once one developer "checks out" a file, others can read that file, but no one else may change that file until that developer "checks in" the updated version (or cancels the checkout).

File locking has both merits and drawbacks. It can provide some protection against difficult merge conflicts when a user is making radical changes to many sections of a large file (or group of files). If the files are left exclusively locked for too long, other developers may be tempted to bypass the revision control software and change the files locally, forcing a difficult manual merge when the other changes are finally checked in. In a large organization, files can be left "checked out" and locked and forgotten about as developers move between projects - these tools may or may not make it easy to see who has a file checked out.

Version merging

Most version control systems allow multiple developers to edit the same file at the same time. The first developer to "check in" changes to the central repository always succeeds. The system may provide facilities to merge further changes into the central repository, and preserve the changes from the first developer when other developers check in.

Merging two files can be a very delicate operation, and usually possible only if the data structure is simple, as in text files. The result of a merge of two image files might not result in an image file at all. The second developer checking in files will need to take care with the merge, to make sure that the changes are compatible and that the merge operation does not introduce its own logic errors within the files. These problems limit the availability of automatic or semi-automatic merge operations mainly to simple text-based documents, unless a specific merge plugin is available for the file types.

The concept of a *reserved edit* can provide an optional means to explicitly lock a file for exclusive write access, even when a merging capability exists.

Baselines, labels and tags

Most revision control tools will use only one of these similar terms (baseline, label, tag) to refer to the action of identifying a snapshot ("label the project") or the record of the snapshot ("try it with baseline *X*"). Typically only one of the terms *baseline*, *label*, or *tag* is used in documentation or discussion; they can be considered synonyms.

In most projects, some snapshots are more significant than others, such as those used to indicate published releases, branches, or milestones.

When both the term *baseline* and either of *label* or *tag* are used together in the same context, *label* and *tag* usually refer to the mechanism within the tool of identifying or making the record of the snapshot, and *baseline* indicates the increased significance of any given label or tag.

Most formal discussion of configuration management uses the term *baseline*.

Distributed revision control

Distributed revision control systems (DRCS) take a peer-to-peer approach, as opposed to the client–server approach of centralized systems. Rather than a single, central repository on which clients synchronize, each peer's working copy of the codebase is a bona-fide repository.[12] Distributed revision control conducts synchronization by exchanging patches (change-sets) from peer to peer. This results in some important differences from a centralized system:

- No canonical, reference copy of the codebase exists by default; only working copies.
- Common operations (such as commits, viewing history, and reverting changes) are fast, because there is no need to communicate with a central server.[1]:7

Rather, communication is only necessary when pushing or pulling changes to or from other peers.

- Each working copy effectively functions as a remote backup of the codebase and of its change-history, providing inherent protection against data loss.[1]:4

Best practices

Following best practices is necessary to obtain the full benefits of version control. Best practice may vary by version control tool and the field to which version control is applied. The generally accepted best practices in software development include: making small/incremental changes; making commits which involve only one task or fix -- a corollary to this is to commit only code which works and does not knowingly break existing functionality; using branching to complete functionality before release; writing clear and descriptive commit messages, make what why and how clear in either the commit description or the code; and using a consistent branching strategy.[13] Other best software development practices such as code review and automated regression testing may assist in the following of version control best practices.

Costs and benefits

Costs and benefits will vary dependent upon the version control tool chosen and the field in which it is applied. This section speaks to the field of software development, where version control is widely applied.

Costs

In addition to the costs of licensing the version control software, using version control requires time and effort. The concepts underlying version control must be understood and the technical particulars required to operate the version control software chosen must be learned. Version control best practices must be learned and integrated into the organization's existing software development practices. Management effort may be required to maintain the discipline needed to follow best practices in order to obtain useful benefit.

Benefits

Allows for reverting changes

A core benefit is the ability to keep history and revert changes, allowing the developer to easily undo changes. This gives the developer more opportunity to experiment, eliminating the fear of breaking existing code.[14]

Branching simplifies deployment, maintenance and development

Branching assists with deployment. Branching and merging, the production, packaging, and labeling of source code patches and the easy application of patches to code bases, simplifies the maintenance and concurrent development of the multiple code bases associated with the various stages of the deployment process; development, testing, staging, production, etc.[15]

Damage mitigation, accountability and process and design improvement

There can be damage mitigation, accountability, process and design improvement, and other benefits associated with the record keeping provided by version control, the tracking of who did what, when, why, and how.[16]

When bugs arise, knowing what was done when helps with damage mitigation and recovery by assisting in the identification of what problems exist, how long they have existed, and determining problem scope and solutions.[17] Previous versions can be installed and tested to verify conclusions reached by examination of code and commit messages.[18]

Simplifies debugging

Version control can greatly simplify debugging. The application of a test case to multiple versions can quickly identify the change which introduced a bug.[19] The developer need not be familiar with the entire code base and can focus instead on the code that introduced the problem.

Improves collaboration and communication

Version control enhances collaboration in multiple ways. Since version control can identify conflicting changes, i.e. incompatible changes made to the same lines of code, there is less need for coordination among developers.[20]

The packaging of commits, branches, and all the associated commit messages and version labels, improves communication between developers, both in the moment and over time.[21] Better communication, whether instant or deferred, can improve the code review process, the testing process, and other critical aspects of the software development process.

Integration

Some of the more advanced revision-control tools offer many other facilities, allowing deeper integration with other tools and software-engineering processes.

Integrated development environment

Plugins are often available for IDEs such as Oracle JDeveloper, IntelliJ IDEA, Eclipse, Visual Studio, Delphi, NetBeans IDE, Xcode, and GNU Emacs (via vc.el). Advanced research prototypes generate appropriate commit messages.[22]

Common terminology

Terminology can vary from system to system, but some terms in common usage include:[23]

Baseline

An approved revision of a document or source file to which subsequent changes can be made. See baselines, labels and tags.

Blame

A search for the author and revision that last modified a particular line.

Branch

A set of files under version control may be *branched* or *forked* at a point in time so that, from that time forward, two copies of those files may develop at different speeds or in different ways independently of each other.

Change

A *change* (or *diff*, or *delta*) represents a specific modification to a document under version control. The granularity of the modification considered a change varies between version control systems.

Change list

On many version control systems with atomic multi-change commits, a *change list* (or *CL*), *change set*, *update*, or *patch* identifies the set of *changes* made in a single commit. This can also represent a sequential view of the source code, allowing the examination of source as of any particular changelist ID.

Checkout

To *check out* (or *co*) is to create a local working copy from the repository. A user may specify a specific revision or obtain the latest. The term 'checkout' can also be used as a noun to describe the working copy. When a file has been checked out from a shared file server, it cannot be edited by other users. Think of it like a hotel, when you check out, you no longer have access to its amenities.

Clone

Cloning means creating a repository containing the revisions from another repository. This is equivalent to *push*ing or *pull*ing into an empty (newly initialized) repository. As a noun, two repositories can be said to be *clone*s if they are kept synchronized, and contain the same revisions.

Commit (noun)

In version control software, a changeset (also known as commit[24] and revision[25][26]) is a set of alterations packaged together, along with meta-information about the alterations. A changeset describes the exact differences between two successive versions in the version control system's repository of changes. Changesets are typically treated as an atomic unit, an indivisible set, by version control systems. This is one synchronization model.[27][28]

Commit (verb)

To *commit* (*check in*, *ci* or, more rarely, *install*, *submit* or *record*) is to write or merge the changes made in the working copy back to the repository. A commit contains metadata, typically the author information and a commit message that describes the change.

Commit message

A short note, written by the developer, stored with the commit, which describes the commit. Ideally, it records why the modification was made, a description of the modification's effect or purpose, and non-obvious aspects of how the change works.

Conflict

A conflict occurs when different parties make changes to the same document, and the system is unable to reconcile the changes. A user must *resolve* the conflict by combining the changes, or by selecting one change in favour of the other.

Delta compression

Most revision control software uses delta compression, which retains only the differences between successive versions of files. This allows for more efficient storage of many different versions of files.

Dynamic stream

A stream in which some or all file versions are mirrors of the parent stream's versions.

Export

Exporting is the act of obtaining the files from the repository. It is similar to *checking out* except that it creates a clean directory tree without the version-control metadata used in a working copy. This is often used prior to publishing the contents, for example.

Fetch

See *pull*.

Forward integration

The process of merging changes made in the main *trunk* into a development (feature or team) branch.

Head

Also sometimes called *tip*, this refers to the most recent commit, either to the trunk or to a branch. The trunk and each branch have their own head, though HEAD is sometimes loosely used to refer to the trunk.[29]

Import

Importing is the act of copying a local directory tree (that is not currently a working copy) into the repository for the first time.

Initialize

To create a new, empty repository.

Interleaved deltas

Some revision control software uses Interleaved deltas, a method that allows storing the history of text based files in a more efficient way than by using Delta compression.

Label

See *tag*.

Locking

When a developer *locks* a file, no one else can update that file until it is unlocked. Locking can be supported by the version control system, or via informal communications between developers (aka *social locking*).

Mainline

Similar to *trunk*, but there can be a mainline for each branch.

Merge

A *merge* or *integration* is an operation in which two sets of changes are applied to a file or set of files. Some sample scenarios are as follows:

- A user, working on a set of files, *updates* or *syncs* their working copy with changes made, and checked into the repository, by other users.[30]
- A user tries to *check in* files that have been updated by others since the files were *checked out*, and the *revision control software* automatically merges the files (typically, after prompting the user if it should proceed with the automatic merge, and in some cases only doing so if the merge can be clearly and reasonably resolved).
- A *branch* is created, the code in the files is independently edited, and the updated branch is later incorporated into a single, unified *trunk*.
- A set of files is *branched*, a problem that existed before the branching is fixed in one branch, and the fix is then merged into the other branch. (This type of selective merge is sometimes known as a *cherry pick* to distinguish it from the complete merge in the previous case.)

Promote

The act of copying file content from a less controlled location into a more controlled location. For example, from a user's workspace into a repository, or from a stream to its parent.[31]

Pull, push

Copy revisions from one repository into another. *Pull* is initiated by the receiving repository, while *push* is initiated by the source. *Fetch* is sometimes used as a synonym for *pull*, or to mean a *pull* followed by an *update*.

Pull request

Contributions to a source code repository that uses a distributed version control system are commonly made by means of a pull request, also known as a merge request.[32] The contributor requests that the project maintainer *pull* the source code change, hence the name "pull request". The maintainer has to *merge* the pull request if the contribution should become part of the source base.[33]

The developer creates a pull request to notify maintainers of a new change; a comment thread is associated with each pull request. This allows for focused discussion of code changes. Submitted pull requests are visible to anyone with repository access. A pull request can be accepted or rejected by maintainers.[34]

Once the pull request is reviewed and approved, it is merged into the repository. Depending on the established workflow, the code may need to be tested before being included into official release. Therefore, some projects contain a special branch for merging untested pull requests.[33][35] Other projects run an automated test suite on every pull request, using a continuous integration tool, and the reviewer checks that any new code has appropriate test coverage.

Repository

In version control systems, a repository is a data structure that stores metadata for a set of files or directory structure.[36] Depending on whether the version control system in use is distributed, like Git or Mercurial, or centralized, like Subversion, CVS, or Perforce, the whole set of information in the repository may be duplicated on every user's system or may be maintained on a single server.[37] Some of the metadata that a repository contains includes, among other things, a historical record of changes in the repository, a set of commit objects, and a set of references to commit objects, called *heads*.

Resolve

The act of user intervention to address a conflict between different changes to the same document.

Reverse integration

The process of merging different team branches into the main trunk of the versioning system.

Revision and version

A version is any change in form. In SVK, a Revision is the state at a point in time of the entire tree in the repository.

Share

The act of making one file or folder available in multiple branches at the same time. When a shared file is changed in one branch, it is changed in other branches.

Stream

A container for branched files that has a known relationship to other such containers. Streams form a hierarchy; each stream can inherit various properties (like versions, namespace, workflow rules, subscribers, etc.) from its parent stream.

Tag

A *tag* or *label* refers to an important snapshot in time, consistent across many files. These files at that point may all be tagged with a user-friendly, meaningful name or revision number. See baselines, labels and tags.

Trunk

The trunk is the unique line of development that is not a branch (sometimes also called Baseline, Mainline or Master[38][39])

Update

An *update* (or *sync*, but *sync* can also mean a combined *push* and *pull*) merges changes made in the repository (by other people, for example) into the local *working copy*. *Update* is also the term used by some CM tools (CM+, PLS, SMS) for the change package concept (see *changelist*). Synonymous with *checkout* in revision control systems that require each repository to have exactly one working copy (common in distributed systems)

Unlocking

Releasing a lock.

Working copy

The *working copy* is the local copy of files from a repository, at a specific time or revision. All work done to the files in a repository is initially done on a working copy, hence the name. Conceptually, it is a *sandbox*.

See also

- Change control
- Changelog
- Blockchain
- Comparison of version-control software
- Comparison of source-code-hosting facilities
- Data version control
- Distributed version control
- List of version-control software
- Non-linear editing
- Software configuration management
- Software versioning
- Versioning file system

Notes

1. In this case, edit buffers are a secondary form of working copy, and not referred to as such.
2. In principle two revisions can have identical timestamp, and thus cannot be ordered on a line. This is generally the case for separate repositories, though is also possible for simultaneous changes to several branches in a single repository. In these cases, the revisions can be thought of as a set of separate lines, one per repository or branch (or branch within a repository).
3. The revision or repository "tree" should not be confused with the directory tree of files in a working copy.
4. If a new branch is based on HEAD, then topologically HEAD is no longer a tip, since it has a child.
5. "Mainline" can also refer to the main path in a separate branch.

References

1. O'Sullivan, Bryan (2009). *Mercurial: the Definitive Guide* (http://hgbook.red-bean.com/read/). Sebastopol: O'Reilly Media, Inc. ISBN 978-0-596-55547-4. Archived (https://web.archive.org/web/20191208075704/http://hgbook.red-bean.com/read/) from the original on 8 December 2019. Retrieved 4 September 2015.
2. "Google Docs", *See what's changed in a file* (https://support.google.com/docs/answer/190843), Google Inc., archived (https://web.archive.org/web/20221006115757/http://support.google.com/docs/answer/190843) from the original on 2022-10-06, retrieved 2021-04-21.
3. Scott, Chacon; Straub, Ben (2014). *Pro Git Second Edition* (https://git-scm.com/book/en/v2). United States: Apress. p. 14. Archived (https://web.archive.org/web/20151225223054/http://git-scm.com/book/en/v2) from the original on 2015-12-25. Retrieved 2022-02-19.

4. Goetz, Martin (May 3, 2002). "An Interview with Martin Goetz" (https://conservancy.umn.edu/handle/11299/107328) (Interview). Interviewed by Jeffrey R. Yost. Washington, D.C.: Charles Babbage Institute, University of Minnesota. pp. 5–7. Archived (https://web.archive.org/web/20240926164955/https://conservancy.umn.edu/items/0277706b-af0f-4a02-84d6-ff74b1d410aa) from the original on September 26, 2024. Retrieved August 17, 2023.
5. Piscipo, Joseph (May 3, 2002). "An Interview with Joseph Piscopo" (https://conservancy.umn.edu/handle/11299/107601) (Interview). Interviewed by Thomas Haigh. Washington, D.C.: Charles Babbage Institute, University of Minnesota. pp. 3, 5, 12–13. Archived (https://web.archive.org/web/20240926164956/https://conservancy.umn.edu/items/9e1d3842-5a3b-495f-ae5a-d6aa3baaf109) from the original on September 26, 2024. Retrieved August 17, 2023.
6. Rochkind, Marc J. (1975). "The Source Code Control System" (https://web.archive.org/web/20200930100148/https://basepath.com/aup/talks/SCCS-Slideshow.pdf) (PDF). *IEEE Transactions on Software Engineering*. **SE-1** (4): 364–370. Bibcode:1975ITSEn...1..364R (https://ui.adsabs.harvard.edu/abs/1975ITSEn...1..364R). doi:10.1109/TSE.1975.6312866 (https://doi.org/10.1109%2FTSE.1975.6312866). S2CID 10006076 (https://api.semanticscholar.org/CorpusID:10006076). Archived from the original (https://basepath.com/aup/talks/SCCS-Slideshow.pdf) (PDF) on 2020-09-30. Retrieved 2020-09-03.
7. Tichy, Walter F. (1985). "Rcs — a system for version control" (https://docs.lib.purdue.edu/cgi/viewcontent.cgi?article=1393&context=cstech). *Software: Practice and Experience*. **15** (7): 637–654. doi:10.1002/spe.4380150703 (https://doi.org/10.1002%2Fspe.4380150703). ISSN 0038-0644 (https://search.worldcat.org/issn/0038-0644). S2CID 2605086 (https://api.semanticscholar.org/CorpusID:2605086). Archived (https://web.archive.org/web/20240926164944/https://docs.lib.purdue.edu/cgi/viewcontent.cgi?article=1393&context=cstech) from the original on 2024-09-26. Retrieved 2023-12-28.
8. Collins-Sussman, Ben; Fitzpatrick, BW; Pilato, CM (2004), *Version Control with Subversion* (https://archive.org/details/versioncontrolwi00coll), O'Reilly, ISBN 0-596-00448-6
9. Loeliger, Jon; McCullough, Matthew (2012). *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development* (https://books.google.com/books?id=qIucp61eqAwC). O'Reilly Media. pp. 2–5. ISBN 978-1-4493-4504-4. Archived (https://web.archive.org/web/20240926164947/https://books.google.com/books?id=qIucp61eqAwC) from the original on 2024-09-26. Retrieved 2023-03-22.
10. For Engineering drawings, see Whiteprint#Document control, for some of the manual systems in place in the twentieth century, for example, the *Engineering Procedures* of Hughes Aircraft, each revision of which required approval by Lawrence A. Hyland; see also the approval procedures instituted by the U.S. government.
11. Smart, John Ferguson (2008). *Java Power Tools* (https://books.google.com/books?id=YoTvBpKEx5EC&q=cvs+doesn%27t+support+atomic+commit&pg=PA301). "O'Reilly Media, Inc.". p. 301. ISBN 978-1-4919-5454-6. Archived (https://web.archive.org/web/20240926164948/https://books.google.com/books?id=YoTvBpKEx5EC&q=cvs+doesn%27t+support+atomic+commit&pg=PA301#v=snippet&q=cvs%20doesn't%20support%20atomic%20commit&f=false) from the original on 26 September 2024. Retrieved 20 July 2019.
12. Wheeler, David. "Comments on Open Source Software / Free Software (OSS/FS) Software Configuration Management (SCM) Systems" (https://web.archive.org/web/20201109022758/http://www.dwheeler.com/essays/scm.html). Archived from the original (http://www.dwheeler.com/essays/scm.html) on November 9, 2020. Retrieved May 8, 2007.
13. GitLab. "What are Git version control best practices?" (https://about.gitlab.com/topics/version-control/version-control-best-practices/). *gitlab.com*. Retrieved 2022-11-11.
14. Alessandro Picarelli (2020-11-17). "The cost of not using version control" (https://www.claytex.com/tech-blog/the-cost-of-not-using-version-control/). Archived (https://web.archive.org/web/20221119184145/https://www.claytex.com/tech-blog/the-cost-of-not-using-version-control/) from the original on 2022-11-19. Retrieved 2022-11-18. "In terms of man hours it's going to cost you 6 to 48 times what it would have cost you using version control, and that's for rewinding a couple of models for a single developer."

15. Irma Azarian (2023-06-14). "A Review of Software Version Control: Systems, Benefits, and Why it Matters" (https://www.seguetech.com/a-review-of-software-version-control-systems-benefits-and-why-it-matters/). Archived (https://web.archive.org/web/20240926165505/https://www.seguetech.com/a-review-of-software-version-control-systems-benefits-and-why-it-matters/) from the original on 2024-09-26. Retrieved 2022-11-18. "Version control systems allow you to compare files, identify differences, and merge the changes if needed prior to committing any code. Versioning is also a great way to keep track of application builds by being able to identify which version is currently in development, QA, and production."
16. ReQtest (2020-10-26). "What Are The Benefits Of Version Control?" (https://reqtest.com/requirements-blog/what-are-benefits-of-version-control/). Archived (https://web.archive.org/web/20221122023907/https://reqtest.com/requirements-blog/what-are-benefits-of-version-control/) from the original on 2022-11-22. Retrieved 2022-11-21. "The history of the document provides invaluable information about the author and the date of editing. It also gives on the purpose of the changes made. It will have an impact on the developer that works on the latest version as it will help to solve problems experienced in earlier versions. The ability to identify the author of the document enables the current team to link the documents to specific contributors. This, in turn, enables the current team to uncover patterns that can help with fixing bugs. This will help to improve the overall functionality of the software."
17. Chiradeep BasuMallick (2022-10-06). "What Is Version Control? Meaning, Tools, and Advantages" (https://www.spiceworks.com/tech/devops/articles/what-is-version-control/). Archived (https://web.archive.org/web/20221119184200/https://www.spiceworks.com/tech/devops/articles/what-is-version-control/) from the original on 2022-11-19. Retrieved 2022-11-18. "Software teams can understand the evolution of a solution by examining prior versions through code reviews."
18. Chiradeep BasuMallick (2022-10-06). "What Is Version Control? Meaning, Tools, and Advantages" (https://www.spiceworks.com/tech/devops/articles/what-is-version-control/). Archived (https://web.archive.org/web/20221119184200/https://www.spiceworks.com/tech/devops/articles/what-is-version-control/) from the original on 2022-11-19. Retrieved 2022-11-18. "If an error is made, developers can go back in time and review prior iterations of the code to remedy the mistake while minimizing disturbance for all team members."
19. Chacon, Scott; Straub, Ben (2022-10-03). "Pro Git" (https://git-scm.com/book/en/v2/Appendix-C%3A-Git-Commands-Debugging) (Version 2.1.395-2-g27002dd ed.). Apress. Archived (https://web.archive.org/web/20240926165502/https://git-scm.com/book/en/v2/Appendix-C%3A-Git-Commands-Debugging) from the original on 2024-09-26. Retrieved 2022-11-18. "The git bisect tool is an incredibly helpful debugging tool used to find which specific commit was the first one to introduce a bug or problem by doing an automatic binary search."
20. Irma Azarian (2023-06-14). "A Review of Software Version Control: Systems, Benefits, and Why it Matters" (https://www.seguetech.com/a-review-of-software-version-control-systems-benefits-and-why-it-matters/). Archived (https://web.archive.org/web/20240926165505/https://www.seguetech.com/a-review-of-software-version-control-systems-benefits-and-why-it-matters/) from the original on 2024-09-26. Retrieved 2022-11-18. "Versioning is a priceless process, especially when you have multiple developers working on a single application, because it allows them to easily share files. Without version control, developers will eventually step on each other's toes and overwrite code changes that someone else may have completed without even realizing it. Using these systems allows you to check files out for modifications, then, during check-in, if the files have been changed by another user, you will be alerted and allowed to merge them."
21. Jesse Phillips (2019-01-21) [2018-12-12]. "Git is a Communication tool" (https://dev.to/jessekphillips/git-is-a-communication-tool--2j9k). Archived (https://web.archive.org/web/20221119184143/https://dev.to/jessekphillips/git-is-a-communication-tool--2j9k) from the original on 2022-11-19. Retrieved 2022-11-18. "There are continued discussions on using rebase, merge, and/or squash. I want to bring focus to the point of all these choices, communicating."

22. Cortes-Coy, Luis Fernando; Linares-Vasquez, Mario; Aponte, Jairo; Poshyvanyk, Denys (2014). "On Automatically Generating Commit Messages via Summarization of Source Code Changes". *2014 IEEE 14th International Working Conference on Source Code Analysis and Manipulation*. IEEE. pp. 275–284. doi:10.1109/scam.2014.14 (https://doi.org/10.1109%2Fscam.2014.14). ISBN 978-1-4799-6148-1. S2CID 360545 (https://api.semanticscholar.org/CorpusID:360545).
23. Wingerd, Laura (2005). *Practical Perforce* (https://web.archive.org/web/20071221132707/http://safari.oreilly.com/0596101856). O'Reilly. ISBN 0-596-10185-6. Archived from the original (http://safari.oreilly.com/0596101856) on 2007-12-21. Retrieved 2006-08-27.
24. changeset in the gitglossary (https://git-scm.com/docs/gitglossary#Documentation/gitglossary.txt-aiddefchangesetachangeset)
25. revision in the gitglossary (https://git-scm.com/docs/gitglossary#def_revision)
26. UnderstandingMercurial - Mercurial (https://mercurial.selenic.com/wiki/UnderstandingMercurial#Committing_changes)
27. Mercurial: ChangeSet (http://mercurial.selenic.com/wiki/ChangeSet) Archived (https://web.archive.org/web/20100115230528/http://mercurial.selenic.com/wiki/ChangeSet) January 15, 2010, at the Wayback Machine
28. "Version Control System Comparison" (https://web.archive.org/web/20090321051311/http://better-scm.berlios.de/comparison/comparison.html). Better SCM Initiative. Archived from the original (http://better-scm.berlios.de/comparison/comparison.html) on 2009-03-21.
29. Gregory, Gary (February 3, 2011). "Trunk vs. HEAD in Version Control Systems" (http://garygregory.wordpress.com/2011/02/03/trunk-vs-head-in-version-control-systems/). *Java, Eclipse, and other tech tidbits*. Archived (https://web.archive.org/web/20200920130412/https://garygregory.wordpress.com/2011/02/03/trunk-vs-head-in-version-control-systems/) from the original on 2020-09-20. Retrieved 2012-12-16.
30. Collins-Sussman, Fitzpatrick & Pilato 2004, 1.5: SVN tour cycle resolve (http://svnbook.red-bean.com/en/1.5/svn.tour.cycle.html#svn.tour.cycle.resolve): 'The G stands for merGed, which means that the file had local changes to begin with, but the changes coming from the repository didn't overlap with the local changes.'
31. *Concepts Manual* (Version 4.7 ed.). Accurev. July 2008.
32. Sijbrandij, Sytse (29 September 2014). "GitLab Flow" (https://about.gitlab.com/2014/09/29/gitlab-flow/). *GitLab*. Retrieved 4 August 2018.
33. Johnson, Mark (8 November 2013). "What is a pull request?" (http://oss-watch.ac.uk/resources/pullrequest). *Oaawatch*. Retrieved 27 March 2016.
34. "Using pull requests" (https://help.github.com/articles/using-pull-requests/). GitHub. Retrieved 27 March 2016.
35. "Making a Pull Request" (https://www.atlassian.com/git/tutorials/making-a-pull-request). Atlassian. Retrieved 27 March 2016.
36. "SVNBook" (http://svnbook.red-bean.com/en/1.7/svn.basic.version-control-basics.html#svn.basic.repository). Retrieved 2012-04-20.
37. "Version control concepts and best practices" (https://homes.cs.washington.edu/~mernst/advice/version-control.html). 2018-03-03. Archived (https://web.archive.org/web/20200427074258/https://homes.cs.washington.edu/~mernst/advice/version-control.html) from the original on 2020-04-27. Retrieved 2020-07-10.
38. Wallen, Jack (2020-09-22). "GitHub to replace master with main starting in October: What developers need to do now" (https://www.techrepublic.com/article/github-to-replace-master-with-main-starting-in-october-what-developers-need-to-know/). *TechRepublic*. Archived (https://web.archive.org/web/20210208011318/https://www.techrepublic.com/article/github-to-replace-master-with-main-starting-in-october-what-developers-need-to-know/) from the original on 2021-02-08. Retrieved 2022-04-24.

39. Heinze, Carolyn (2020-11-24). "Why GitHub renamed its master branch to main" (https://www.theserverside.com/feature/Why-GitHub-renamed-its-master-branch-to-main). *TheServerSide*. Archived (https://web.archive.org/web/20220526125637/https://www.theserverside.com/feature/Why-GitHub-renamed-its-master-branch-to-main) from the original on 2022-05-26. Retrieved 2022-04-24.

External links

- "Visual Guide to Version Control", *Better explained* (http://betterexplained.com/articles/a-visual-guide-to-version-control/).
- Sink, Eric, "Source Control", *SCM* (http://www.ericsink.com/scm/source_control.html) (how-to). The basics of version control.